Fast and Fourier

IIT Bhubaneswar

Tushar Joshi, Soham Chakraborty, Arihant Garg

March 7, 2025

Contents

1 Strings

1.1 manacher

```
vector<vector<int>> d;
// d[0] contains half the length of max palindrome of
// odd length with centre at i d[1] contains half the
// length of max palindrome of even length with right
// centre at i

void manacher(string s) {
  int n = s.size();
  d.resize(n, vector<int>(2));
  for(int t = 0; t < 2; t++) {
    int l = 0, r = -1, j;
    for(int i = 0; i < n; i++) {
      j = (i > r) x ? 1
            : min(d[l + r - i + t][t], r - i + t) + 1;

      while(i + j - t < n && i - j >= 0
            && s[i + j - t] == s[i - j])
        j++;

      d[i][t] = --j;
      if(i + j + t > r) {
        l = i - j;
        r = i + j - t;
      }
    }
  }
}
```

1.2 aho_corasick

```
const static int K = 26;
// Will change if input is not just lowercase alphabets
struct Vertex {
  int next[K];
  int leaf = 0;
  // It actually denotes number of leafs reachable
  // from current vertexes using links.
  int p = -1;
  char pch;
  int link = -1;

  Vertex(int p = -1, char ch = '$') : p(p), pch(ch) {
    fill(begin(next), end(next), -1);
  }
};
vector<Vertex> t(1);
// Automation is stored in form of vector.

// Add String s to the automaton.
void add_s(string const &s) {
  int v = 0;
  for(char ch : s) {
    int c = ch - 'a';
    if(t[v].next[c] == -1) {
      t[v].next[c] = t.size();
      t.emplace_back(v, ch);
    }
    v = t[v].next[c];
  }
  t[v].leaf += 1;
}

// Forward declaration of functions
int go(int v, char ch);

// gets the link from vertex v.
int get_link(int v) {
  if(t[v].link == -1) {
    if(v == 0 || t[v].p == 0) t[v].link = 0;
    else
      t[v].link = go(get_link(t[v].p), t[v].pch);
  }
  return t[v].link;
}

int go(int v, char ch) {
  int c = ch - 'a';
  // May change if not lowercase alphabet

  if(t[v].next[c] == -1)
    t[v].next[c] = v == 0 ? 0 : go(get_link(v), ch);

  return t[v].next[c];
}

// To calculate links and leafs(exit link) for nodes.
void bfs() {
  queue<int> order;
  order.push(0);
  while(!order.empty()) {
    int cur = order.front();
    order.pop();
    t[cur].link = get_link(cur);
    t[cur].leaf += t[t[cur].link].leaf;
    for(int i = 0; i < K; ++i) {
      if(t[cur].next[i] != -1) {
        order.push(t[cur].next[i]);
      }
    }
  }
}
```

1.3 suffix_array

```
// Time Complexity: O(nlogn)

void count_sort(vector<int> &p,vector<int> &c){ //
    Sorts values in p by keys in c i.e, if c[p[i]] <
    c[p[j]] then i appears before j in p.
    int n = p.size();
    vector<int> cnt(n);
    for(auto x : c) cnt[x]++;
    vector<int> p_new(n),pos(n);
    pos[0] = 0;
    for(int i=1;i<n;++i) pos[i] = pos[i-1] + cnt[i-1];
    for(auto x : p){
        int i = c[x];
        p_new[pos[i]] = x;
        pos[i]++;
    }
    p = p_new;
}

// Returns sorted suffix_array of size (n+1) with first
    element = n (empty suffix).
vector<int> suffix_array(string s){
```

```
    s += '$';
    int n = s.size();
    vector<int> p(n),c(n); // p stores suffix array and
        c stroes its equivalence class
    {
        // k = 0 phase
        vector<pair<char,int>> a(n);
        for(int i=0;i<n;++i) a[i] = {s[i],i};
        sort(all(a));
        for(int i=0;i<n;++i) p[i] = a[i].S;
        c[p[0]] = 0;
        for(int i=1;i<n;++i){
            if(a[i].F == a[i-1].F) c[p[i]] = c[p[i-1]];
            else c[p[i]] = c[p[i-1]] + 1;
        }
    }
    int k=0;
    while((1<<k) < n ){ // k -> k + 1
        // We require to sort p by {c[i], c[i+(1<<k)]}
            value. So we use radix sort: Sort the
            second element of pair then count sort
            first element in a stable fashion.
        // Sort by second element: p[i] -= (1<<k). As
            p[i]'s are already sorted by c[i] values.
        for(int i = 0; i < n ; ++i) p[i] = (p[i] -
            (1<<k) + n )%n;
        count_sort(p,c);

        vector<int> c_new(n);
        c_new[p[0]] = 0;
        for(int i=1;i<n;++i){
            pii prev = {c[p[i-1]] , c[(p[i-1] +
                (1<<k))%n] } ;
            pii now = {c[p[i]] , c[(p[i] + (1<<k))%n] } ;
            if( now == prev) c_new[p[i]] = c_new[p[i-1]];
            else c_new[p[i]] = c_new[p[i-1]] + 1;
        }
        c = c_new;
        ++k;
    }
    return p;
}


vector<int> lcp_array(vector<int> &p,vector<int>
    &c,string s){
    int n = p.size();
    vector<int> lcp(n-1);
    int k=0;
    for(int i=0;i<n-1;++i){
        int pi = c[i];
        int j = p[pi-1];
        //lcp[i] = lcp[s[i...],s[j...]]
        while(s[i+k] == s[j+k]) ++k;
        lcp[pi-1] = k;
        k = max(k-1,0);
    }
    return lcp;
}
// Finds lcp using p and
// c array defined in suffix array
```

2 Trees

2.1 hld

```
// Build, query, update, node::combine
struct HLD {
  vector<int> ord;
  vector<int> pos;
  vector<int> head;
  vector<int> par;
  vector<int> depth;
  vector<int> heavy;
  SegmentTree st;

  void build(vector<vector<int>> &g, vector<int> &a) {
    int n = g.size() - 1;
    pos.assign(n + 1, 0), head.assign(n + 1, 0),
      par.assign(n + 1, 0), depth.assign(n + 1, 0),
      heavy.assign(n + 1, 0);

    function<int(int, int)> dfs = [&](int v, int p) {
      int sz = 1;
      int mxcsz = 0;
      for(auto c : g[v]) {
        if(c != p) {
          par[c] = v;
          depth[c] = depth[v] + 1;

          int csz = dfs(c, v);

          sz += csz;
          if(csz > mxcsz) {
            heavy[v] = c;
            mxcsz = csz;
          }
        }
      }
      return sz;
    };

    function<void(int, int, int)> decomp
      = [&](int v, int p, int h) {
        head[v] = h;
        ord.push_back(a[v]);
        pos[v] = ord.size() - 1;

        if(heavy[v]) decomp(heavy[v], v, h);
        for(auto c : g[v]) {
          if(c != p && c != heavy[v]) {
            decomp(c, v, c);
          }
        }
      };

    dfs(1, 0);
    decomp(1, 0, 1);
    st.build(ord);
  }

  int query(int u, int v) {
    node resu, resv;
    for(; head[u] != head[v]; v = par[head[v]]) {
      if(depth[head[u]] > depth[head[v]]) {
        swap(u, v);
        swap(resu, resv);
      }
      node q = st.query(pos[head[v]], pos[v]);
      resv = node::combine(q, resv);
    }
    if(depth[u] > depth[v]) {
      swap(u, v);
```

```
    swap(resu, resv);
  }

  node q = st.query(pos[u], pos[v]);
  resv = node::combine(q, resv);

  // resv stores query LCA to v
  // resu stores query form LCA to u
  // lower depth side is considered left in each
  // query, process accordingly

  int res; // define combination of lca - node queries
  return res;
}

void update(int u, int v, int x) {
  for(; head[u] != head[v]; v = par[head[v]]) {
    if(depth[head[u]] > depth[head[v]]) {
      swap(u, v);
    }
    st.update(pos[head[v]], pos[v], x);
  }
  if(depth[u] > depth[v]) { swap(u, v); }
  st.update(pos[u], pos[v], x);
}
};
```

2.2 top_tree

```
#define opr operator
struct Data {
  int v;
  Data() : v(0) {}
  Data(int v) : v(v) {}
};

struct Upd {
  int v;
  Upd(int v = 0) : v(v) {}
  bool upd() { return v; }
};

// node combine, upd combine and upd apply definitions
Data opr + (const Data &a, const Data &b) {
  return Data(a.v + b.v);
}
Data &opr += (Data & a, const Data &b) {
  return a = a + b;
}
Upd &opr += (Upd & a, const Upd &b) {
  return a = Upd(max(a.v, b.v));
}
Data &opr += (Data & a, const Upd &b) {
  return a = Data(a.v + b.v);
}

struct node {
  int par, child[4];
  Data path, sub, all, data;
  Upd plazy, slazy;
  bool flip, fake;
  node()
      : par(0), child(), path(), sub(), all(), plazy(),
        slazy(), flip(false), fake(true) {}
  node(int v) : node() {
    data = Data(v);
    path = all = Data(data);
    fake = false;
  }
};
// Splay Tree
struct SplayTree {
  vector<node> t;
  SplayTree(int n) : t(n) {}
  void pushflip(int u) {
    if(!u) return;
    swap(t[u].child[0], t[u].child[1]);
    t[u].flip ^= true;
  }
  void pushpath(int u, const Upd &upd) {
    if(!u || t[u].fake) return;
    t[u].data += upd;
    t[u].path += upd;
    t[u].all = t[u].path + t[u].sub;
    t[u].plazy += upd;
  }
  void pushsub(int u, bool tag, const Upd &upd) {
    if(!u) return;
    t[u].sub += upd;
    t[u].slazy += upd;
    if(!t[u].fake && tag) pushpath(u, upd);
    else
      t[u].all = t[u].path + t[u].sub;
  }
  void push(int u) {
    if(!u) return;
    if(t[u].flip) {
      pushflip(t[u].child[0]);
      pushflip(t[u].child[1]);
      t[u].flip = false;
    }
    if(t[u].plazy.upd()) {
      pushpath(t[u].child[0], t[u].plazy);
      pushpath(t[u].child[1], t[u].plazy);
      t[u].plazy = Upd();
    }
    if(t[u].slazy.upd()) {
      pushsub(t[u].child[0], false, t[u].slazy);
      pushsub(t[u].child[1], false, t[u].slazy);
      pushsub(t[u].child[2], true, t[u].slazy);
      pushsub(t[u].child[3], true, t[u].slazy);
      t[u].slazy = Upd();
    }
  }
  void pull(int u) {
    if(!t[u].fake)
      t[u].path = t[t[u].child[0]].path + t[u].data
                  + t[t[u].child[1]].path;
    t[u].sub
      = t[t[u].child[0]].sub + t[t[u].child[1]].sub
        + t[t[u].child[2]].all + t[t[u].child[3]].all;
    t[u].all = t[u].path + t[u].sub;
  }
  void attach(int u, int d, int v) {
    t[u].child[d] = v;
    t[v].par = u;
    pull(u);
  }
  int dir(int u, int tag) {
    int v = t[u].par;
    return t[v].child[tag] == u    ? tag
          : t[v].child[tag + 1] == u ? tag + 1
```

```
                                          : -1;
  }
  void rotate(int u, int tag) {
    int v = t[u].par, w = t[v].par, du = dir(u, tag),
        dv = dir(v, tag);
    if(dv == -1 && tag == 0) dv = dir(v, 2);
    attach(v, du, t[u].child[du ^ 1]);
    attach(u, du ^ 1, v);
    if(~dv) attach(w, dv, u);
    else
      t[u].par = w;
  }
  void splay(int u, int tag) {
    push(u);
    while(~dir(u, tag) && (tag == 0 ||
         t[t[u].par].fake)) {
      int v = t[u].par, w = t[v].par;
      push(w);
      push(v);
      push(u);
      int du = dir(u, tag), dv = dir(v, tag);
      if(~dv && (tag == 0 || t[w].fake))
        rotate(du == dv ? v : u, tag);
      rotate(u, tag);
    }
  }
};
// Fully Dynamic Tree
struct LinkCut : SplayTree {
  vector<int> fakes;
  LinkCut(int n) : SplayTree(2 * n + 1) {
    for(int i = 1; i <= 2 * n; i++) {
      if(i <= n) t[i].fake = false;
      else
        fakes.push_back(i);
    }
  }
  void add(int u, int v) {
    if(!v) return;
    for(int i = 2; i < 4; i++)
      if(!t[u].child[i]) {
        attach(u, i, v);
        return;
      }
    int w = fakes.back();
    fakes.pop_back();
    attach(w, 2, t[u].child[2]);
    attach(w, 3, v);
    attach(u, 2, w);
  }
  void recPush(int u) {
    if(t[u].fake) recPush(t[u].par);
    push(u);
  }
  void rem(int u) {
    int v = t[u].par;
    recPush(v);
    if(t[v].fake) {
      int w = t[v].par;
      attach(w, dir(v, 2), t[v].child[dir(u, 2) ^ 1]);
      if(t[w].fake) splay(w, 2);
      fakes.push_back(v);
    } else {
      attach(v, dir(u, 2), 0);
    }
    t[u].par = 0;
  }
  int par(int u) {
    int v = t[u].par;
    if(!t[v].fake) return v;
    splay(v, 2);
    return t[v].par;
  }
  int access(int u) {
    int v = u;
    splay(u, 0), add(u, t[u].child[1]), attach(u, 1, 0);
    while(t[u].par) {
      v = par(u);
      splay(v, 0);
      rem(u);
      add(v, t[v].child[1]);
      attach(v, 1, u);
      splay(u, 0);
    }
    return v;
  }
  void reroot(int u) {
    access(u);
    pushflip(u);
  }
  void link(int u, int v) {
    reroot(u);
    access(v);
    add(v, u);
  }
  void cut(int u, int v) {
    reroot(u);
    access(v);
    t[v].child[0] = t[u].par = 0;
    pull(v);
  }
  Data getPath(int u, int v) {
    reroot(u);
    access(v);
    return t[v].path;
  }
  void updatePath(int u, int v, Upd upd) {
    reroot(u);
    access(v);
    pushpath(v, upd);
  }
  Data getSubtree(int v) {
    access(v);
    Data ret = t[v].data;
    for(int i = 2; i < 4; i++)
      ret += t[t[v].child[i]].all;
    return ret;
  }
  void updateSubtree(int v, Upd upd) {
    access(v);
    t[v].data += upd;
    for(int i = 2; i < 4; i++)
      pushsub(t[v].child[i], true, upd);
  }
  int lca(int u, int v) {
    if(u == v) return u;
    access(u);
    int ret = access(v);
    return t[u].par ? ret : 0;
  }
};
// Balanced Binary Search Tree
struct BBST : public SplayTree {
  int n;
```

```
  int root;
  BBST(int n) : SplayTree(n + 5), n(n) {
    for(auto &x : t)
      x.fake = false;
  }
  void build(vector<int> &arr) {
    for(int i = n, v = n; i; i--, v--) {
      t[v].data = Data(arr[i]);
      if(i < n - 1) {
        t[v].child[1] = v + 1, t[v + 1].par = v;
        pull(v);
      }
    }
    t[root = n + 2].child[1] = 1, t[1].par = n + 2;
  }
  int find(int i) {
    int v = t[root].child[1];
    while(true) {
      int l = t[v].child[0], r = t[v].child[1];
      if(t[l].path.n >= i) v = l;
      else if((i -= t[l].path.n) == 1)
        break;
      else
        v = r, --i;
    }
    return v;
  }
  void subtreeSplay(int x, int r) {
    int par = t[r].par, d = dir(r, 0);
    if(~d) t[r].par = 0;
    splay(x, 0);
    if(~d) attach(par, d, x);
  }
  pair<int, int> compressRange(int l, int r) {
    int vl = find(l - 1), vr = find(r + 1);
    subtreeSplay(vl, t[root].child[1]);
    subtreeSplay(vr, t[vl].child[1]);
    return {vl, vr};
  }
  Data query(int l, int r) {
    auto [vl, vr] = compressRange(l, r);
    return t[t[vr].child[0]].path;
  }
  void update(int l, int r, Upd upd) {
    auto [vl, vr] = compressRange(l, r);
    if(int u = t[vr].child[0]) {
      pushpath(u, upd);
      pull(vr);
      pull(vl);
    }
  }
  void flip(int l, int r) {
    auto [vl, vr] = compressRange(l, r);
    if(int u = t[vr].child[0]) pushflip(u);
  }
};
```

2.3 centroid_decomposition

```
const int N=200005; // Change based on constraint
set<int> G[N]; // adjacency list (note that this is
    stored in set, not vector)
int sz[N], pa[N];

int dfs(int u, int p) {
  sz[u] = 1;
  for(auto v : G[u]) if(v != p) {
    sz[u] += dfs(v, u);
  }
  return sz[u];
}
int centroid(int u, int p, int n) {
  for(auto v : G[u]) if(v != p) {
    if(sz[v] > n / 2) return centroid(v, u, n);
  }
  return u;
}
void build(int u, int p) {
  int n = dfs(u, p);
  int c = centroid(u, p, n);
  if(p == -1) p = c;
  pa[c] = p;

  vector<int> tmp(G[c].begin(), G[c].end());
  for(auto v : tmp) {
    G[c].erase(v); G[v].erase(c);
    build(v, c);
  }
}
```

2.4 fenwick_tree

```
struct FenwickTree {
  int n, m;
  vector<vector<int>> bit;
  void build(int _n, int _m) {
    n = _n, m = _m;
    bit.assign(n, vector<int>(m, 0));
  }
  int query(int x, int y) {
    int ans = 0;
    for(; x; x -= x & -x)
      for(int j = y; j; j -= j & -j)
        ans += bit[x][j];
    return ans;
  }
  void update(int x, int y, int val) {
    for(; x < n; x += x & -x)
      for(int j = y; j < m; j += j & -j)
        bit[x][j] += val;
  }
  int query(int x1, int y1, int x2, int y2) {
    return query(x2, y2) - query(x1 - 1, y2) -
        query(x2, y1 - 1)
          + query(x1 - 1, y1 - 1);
  }
};
```

3 NumberTheory

3.1 fast_gcd

```
int gcd(int a, int b) {
  if(!a || !b) return a | b;
  unsigned shift = __builtin_ctzll(a | b);
  a >>= __builtin_ctzll(a);
  do {
    b >>= __builtin_ctzll(b);
```

```
    if(a > b) swap(a, b);
    b -= a;
  } while(b);
  return a << shift;
}
```

3.2 cipolla_sqrt

```
template<int p>
class cipolla_sqrt{
 private:
    static int const phim = p-2;
    static int const phi = p-1;
    static int const phihalf = phi>>1;
    static int const phalf = (p+1)>>1;
 public:
    static int pow(int a,int po=phim){
        int res = 1;
        for(;po;po>>=1,a=normalise(a*1ll*a))
        if(po&1){
            res=normalise(a*1ll*res);
        }
        return res;
    }
    static int normalise(ll x){
        if(x>=p){
            x-=p;if(x>=p)x%=p;return x;
        }
        return x;
    }
    static int get(int x){
        int a = 0,b;
        while(
            pow(b = normalise(a*1ll*a-x+p),
                phihalf)==1
        )++a;
        int po = phalf;
        pair<int,int> v = {a,1}, res = {1,1};
        while(po){
            if(po&1){
                int temp = normalise(res.S*1ll*v.S);
                res.S = normalise((v.F*1ll*res.S)
                        +(res.F*1ll*v.S));
                res.F = normalise((v.F*1ll*res.F)
                        +(b*1ll*temp));
            }
            po>>=1;
            int temp = normalise(v.S*1ll*v.S);
            v.S = normalise((v.F*1ll*v.S)
                            +(v.F*1ll*v.S));
            v.F = normalise((v.F*1ll*v.F)
                            +(b*1ll*temp));
        }
        if(res.F==1)res.F = p - res.F;
        return res.F;
    }
};
```

3.3 floor_sum

```
// f(a,b,c,n) = \sigma ( (a*x + b)/c ) from x=0 to n in
//O(logn) where value is floor of the value.
// a>=0,b>=0,c>0
ll FloorSumAPPositive(ll a, ll b,
  ll c, ll n){
  if(!a) return (b / c) * (n + 1);
  if(a >= c or b >= c) return ( ( n * (n + 1) ) / 2) *
      (a / c)
  + (n + 1) * (b / c) + FloorSumAPPositive(a % c, b %
      c, c, n);
  ll m = (a * n + b) / c;
  return m * n - FloorSumAPPositive(c, c - b - 1, a, m
      - 1);
}

// f(a,b,c,n) = \sigma ( (a*x + b)/c ) from x=0 to n in
//O(logn) where value is floor of the value.
ll FloorSumAP(ll a,ll b,
  ll c, ll n){
    if(c<0){
      c = -c;
      a = -a;
      b = -b;
    }
    ll _qa = (a>=0)?(a/c):((a-c+1)/c);
    ll _qb = (b>=0)?(b/c):((b-c+1)/c);
    ll _a = a-c*_qa;
    ll _b = b-c*_qb;
    return _qa*((n*(n+1))/2)
            +_qb*(n+1)+FloorSumAPPositive(_a,_b,c,n);
}

// f(a,b,c,n) = \sigma ( (a*x + b)/c ) from x=l to r in
//O(logn) where value is floor of the value.
ll FloorSumAP(ll a,ll b,
  ll c, ll l, ll r){
    return FloorSumAP(a,b+a*l,c,r-l);
}
```

3.4 factorizer

```
#define mp make_pair
using i32 = int32_t;
using i64 = int64_t;
using i128 = __int128_t;
namespace factorizer {
constexpr i128 mult(i128 a, i128 b, i128 mod) {
    i128 ans = 0;
    while (b) {
        if (b & 1) {
            ans += a;
            if (ans >= mod) ans -= mod;
        }
        a <<= 1;
        if (a >= mod) a -= mod;
        b >>= 1;
    }
    return ans;
}
constexpr i64 mult(i64 a, i64 b, i64 mod) {
    return (i128)a * b % mod;
}
constexpr i32 mult(i32 a, i32 b, i32 mod) {
    return (i64)a * b % mod;
}

template <typename T>
constexpr T f(T x, T c, T mod) {
```

```
    T ans = mult(x, x, mod) + c;
    if (ans >= mod) ans -= mod;
    return ans;
}

template <typename T>
constexpr T brent(T n, T x0 = 2, T c = 1) {
    T x = x0;
    T g = 1;
    T q = 1;
    T xs, y;

    int m = 128;
    int l = 1;
    while (g == 1) {
        y = x;
        for (int i = 1; i < l; i++) x = f(x, c, n);
        int k = 0;
        while (k < l && g == 1) {
            xs = x;
            for (int i = 0; i < m && i < l - k; i++) {
                x = f(x, c, n);
                q = mult(q, abs(y - x), n);
            }
            g = __gcd(q, n);
            k += m;
        }
        l *= 2;
    }
    if (g == n) {
        do {
            xs = f(xs, c, n);
            g = __gcd(abs(xs - y), n);
        } while (g == 1);
    }
    return g;
}

template <typename T>
T binpower(T base, T e, T mod) {
    T result = 1;
    base %= mod;
    while (e) {
        if (e & 1) result = mult(result, base, mod);
        base = mult(base, base, mod);
        e >>= 1;
    }
    return result;
}

template <typename T>
bool check_composite(T n, T a, T d, int s) {
    T x = binpower(a, d, n);
    if (x == 1 || x == n - 1) return false;
    for (int r = 1; r < s; r++) {
        x = mult(x, x, n);
        if (x == n - 1) return false;
    }
    return true;
};

template <typename T>
bool MillerRabin(T n, const vector<T>& bases) {
    // returns true if n is prime,
    // else returns false.

    if (n < 2) return false;
    int r = 0;
    T d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        r++;
    }

    for (T a : bases) {
        if (n == a) return true;
        if (check_composite(n, a, d, r)) return false;
    }
    return true;
}

template <typename T>
bool IsPrime(T n, const vector<T>& bases) {
    if (n < 2) {
        return false;
    }
    vector<T> small_primes = {2, 3, 5, 7, 11, 13, 17,
     19, 23, 29};
    for (const auto& x : small_primes) {
        if (n % x == 0) {
            return n == x;
        }
    }
    if (n < 31 * 31) {
        return true;
    }

    return MillerRabin(n, bases);
}

bool IsPrime(i64 n) {
    return IsPrime(n, {2, 3, 5, 7, 11, 13, 17, 19, 23,
     29, 31, 37});
}

bool IsPrime(i32 n) { return IsPrime(n, {2, 7, 61}); }

template <typename T>
vector<pair<T, int>> MergeFactors(const vector<pair<T,
    int>>& a,
                              const vector<pair<T,
                                  int>>& b) {
    vector<pair<T, int>> c;
    int i = 0;
    int j = 0;
    while (i < (int)a.size() || j < (int)b.size()) {
        if (i < (int)a.size() && j < (int)b.size() &&
            a[i].first == b[j].first) {
            c.emplace_back(a[i].first, a[i].second +
                b[j].second);
            ++i;
            ++j;
            continue;
        }
        if (j == (int)b.size() ||
            (i < (int)a.size() && a[i].first <
                b[j].first)) {
            c.push_back(a[i++]);
        } else {
            c.push_back(b[j++]);
        }
    }
    return c;
```

```
}

template <typename T>
vector<pair<T, int>> RhoC(T n, T c) {
    if (n <= 1) return {};
    if (!(n & 1))
        return MergeFactors({mp(static_cast<T>(2), 1)},
            RhoC(n >> 1, c));
    if (IsPrime(n)) return {mp(n, 1)};
    T g = brent(n, static_cast<T>(2), c);
    return MergeFactors(RhoC(g, c + 1), RhoC(n / g, c +
        1));
}

template <typename T>
vector<pair<T, int>> Factorize(T n) {
    if (n <= 1) return {};
    return RhoC(n, static_cast<T>(1));
}
} // example factorizer::Factorize(35)
```

3.5 gauss

```
const double EPS = 1e-9; // Only needed for
    non-integral allowed values
const int INF = 2; // it doesn't actually have to be
    infinity or a big number

// If modulo p , modify double to int , and do all
    operations under modulo p

int gauss (vector < vector<double> > a, vector<double>
    & ans) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;

    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;
        if (abs (a[sel][col]) < EPS)
            continue;
        for (int i=col; i<=m; ++i)
            swap (a[sel][i], a[row][i]);
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<=m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
        ++row;
    }

    ans.assign (m, 0);
    for (int i=0; i<m; ++i)
        if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i];
    for (int i=0; i<n; ++i) {
        double sum = 0;
        for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS)
            return 0;
    }

    for (int i=0; i<m; ++i)
        if (where[i] == -1)
            return INF;
    return 1;
}
```

3.6 extended_euclidean

```
// Find a solution to ax + by = 1
int gcd(int a,int b,ll &x,ll &y){
        x=1; y=0;
        ll x1=0,y1=1;
        int a1=a,b1=b;
        while(b1){
        int q=a1/b1;
        tie(x,x1)=make_tuple(x1,x-q*x1);
        tie(y,y1)=make_tuple(y1,y-q*y1);
        tie(a1,b1)=make_tuple(b1,a1-q*b1);
        }
        return a1;
}
bool find_any_solution(int a,int b,int c,ll &x0,ll
     &y0,int &g){
        g=gcd(a,b,x0,y0); // If +ve gcd is required use
            g=abs(g). But after calling
            find_any_solution() function. Not in the
            next line.
        if(!g){
                if(c) return false;
        else{
                        x0=y0=0;
                        return true;
        }
        }
        if(c%g) return false;
        x0 *= c/g;
        y0 *= c/g;
        return true;
}
// Count the number of soln:
// x in range [min_x,max_x], y in range [min_y,max_y]
ll find_all_solutions(int a,int b,int c,int min_x,int
     max_x,int min_y,int max_y){
    ll x,y;
        int g;
    if(min_x>max_x || min_y>max_y ||
        (!find_any_solution(a, b, c, x, y, g))) return
        0;
    if(!a && !b) return
        (max_x-min_x+1)1ll(max_y-min_y+1); // long long
        needed for this case only
    c/=g;
    if(!a) return (min_y<=c &&
        c<=max_y)*(max_x-min_x+1);
    else if(!b) return (min_x<=c &&
        c<=max_x)*(max_y-min_y+1);

        a/=g; b/=g;
    int sign_a=a>0?1:-1, sign_b=b>0?1:-1;

        auto shift=[&](ll k){
```

```
            x+=k*b;
            y-=k*a;
        };


        shift((min_x-x)/b);
    if(x<min_x) shift(sign_b);
    if(x>max_x) return 0;
    ll lx1 = x;

    shift((max_x-x)/b);
    if(x>max_x) shift(-sign_b);
    ll rx1=x;

    shift(-(min_y-y)/a);
    if(y<min_y) shift(-sign_a);
    if(y>max_y) return 0;
    ll lx2=x;

    shift(-(max_y-y)/a);
    if(y>max_y) shift(sign_a);
    ll rx2=x;

    if(lx2>rx2) swap(lx2, rx2);
    ll lx=max(lx1,lx2),rx=min(rx1,rx2);

    if(lx>rx) return 0;
    return (rx-lx)/abs(b)+1;
}
```

4 STL

4.1 bitset

```
// Note: Bitset is not a dynamic array, the size must
    be known at compile time

// Initilisation
bitset<10> b1; // 10 bits initialised to 0
bitset<10> b1(12) // 10 bits initialised to 0000001100
bitset<10> b1("101010"); // 10 bits initialised to
    0000101010

// All operations are 0 - indexed based
// Note that the bits are stored in reverse order
// i.e. b[0] is the rightmost bit

// Set
b1.set(); // Set all bits to 1
b1.set(0); // Set b[0] to 1

// Reset
b1.reset(); // Set all bits to 0
b1.reset(0); // Set b[0] to 0

// Flip
b1.flip(); // Flip all bits
b1.flip(0); // Flip b[0]

// Count 1's
b1.count(); // Count number of 1s

// Get value
b1.test(0); // Get b[0] value
// Bitwise operations
b1 & b2; b1 | b2; b1 ^ b2; ~b1;

// Shift operations
b1 << 1; // Left shift
b1 >> 1; // Right shift

// Comparison
b1 == b2; // Check if equal

// To string
b1.to_string(); // Convert to string

// Any, All, None
b1.any(); // Check if any bit is set
b1.all(); // Check if all bits are set
b1.none(); // Check if no bits are set

// Find first, Find next
// Note that it begins with _ then F (capital) and then
    first/next
b1._Find_first(); // Find first set bit
b1._Find_next(0); // Find next set bit after b[0] (not
    including)
```

4.2 pragmas

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

4.3 bitoperation

```
// Iterate over submasks
for(int sub = mask; sub; sub = (sub - 1) & mask)
  cout << sub << "\n"; // Descending order

int cur = (1 << r) - 1, lowest_bit, ones;
while(cur < (1 << n)) {
  // Printing current binary number with r set bits
  for(int i = n - 1; i >= 0; i--)
    cout << ((cur >> i) & 1);
  cout << "\n";
  lowest_bit = cur & (-cur);
  ones = cur & ~(cur + lowest_bit); // Keep ~ in mind
  if(!r) break;
  else
    cur = cur + lowest_bit + (ones / lowest_bit / 2);
}
```

4.4 pbds

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

#define ordered_set tree<int, null_type, less<int>,\
  rb_tree_tag, tree_order_statistics_node_update>

// strict less than is recommended to avoid problems
// with equality
// order_of_key(T key):number of elements less than key
```

```
// find_by_order(int): gives the 0 based index elem
```

4.5 random

```
mt19937_64 rng(chrono::steady_clock::now().
               time_since_epoch().count());

// call rng() for random number
```

5 Graphs

5.1 max_flow

```
/*
Implementation of Dinic's blocking algorithm
for the maximum flow.
Complexity: V^2 E (faster on real graphs).

please add edges not related to input first
to improve constants

This class accepts a graph
(costructed calling AddEdge) and then solves
the maximum flow problem for any source and sink.

Both directed and undirected graphs are supported.
In case of undirected graphs,
each edge must be added twice.

To compute the maximum flow just call
GetMaxFlowValue(source, sink).
*/

// Flow data type
#define pb push_back
using T = int;
struct Edge {
  int u, v;
  T cap, flow;
};
// Define object globally with limits on number of
// vertices an edges (N, M) as templates
template <int N, int M> struct Dinic {
  T inf = 1e9;
  int esz = 0, n, level[N], ptr[N];
  Edge edge[2 * M];
  vector<int> g[N];
  void init(int _n) { n = _n; }
  void addEdge(int u, int v, int cap) {
    edge[esz] = {u, v, cap, 0};
    edge[esz + 1] = {v, u, 0, 0};
    g[u].pb(esz);
    g[v].pb(esz + 1);
    esz += 2;
  }
  bool bfs(int s, int t) {
    memset(level, -1, N * 4);
    queue<int> q;
    q.push(s), level[s] = 0;
    while(q.size()) {
      int v = q.front();
      q.pop();
      for(int x : g[v]) {
        Edge &e = edge[x];
        if(e.cap > e.flow && level[e.v] == -1)
          level[e.v] = level[e.u] + 1, q.push(e.v);
      }
    }
    return level[t] != -1;
  }
  T dfs(int v, int t, T pf) {
    if(v == t || !pf) return pf;
    T f = 0;
    for(int &i = ptr[v]; i < g[v].size(); i++) {
      int ind = g[v][i];
      Edge &e = edge[ind], &re = edge[ind ^ 1];
      if(level[e.u] != level[e.v] - 1) continue;
      T tf = dfs(e.v, t, min(pf - f, e.cap - e.flow));
      f += tf, e.flow += tf, re.flow -= tf;
      if(f == pf) return f;
    }
    return f;
  }
  T calc(int s, int t) {
    T f = 0;
    while(bfs(s, t)) {
      memset(ptr, 0, N * 4);
      f += dfs(s, t, inf);
    }
    return f;
  }
};
```

5.2 edmond_blossom_unweighted

```
struct edmond {
    int n, m , nE, n_matches,q_n,book_mark;
    vector<int> adj,nxt,go,mate,q,book,type,fa,bel;

    edmond(int n,int m):n(n),m(m){
        nE=0;
        n_matches=0;
        adj.resize(n+1);
        mate.resize(n+1);
        q.resize(n+1);
        book.resize(n+1);
        type.resize(n+1);
        fa.resize(n+1);
        bel.resize(n+1);
        go.resize((m<<1)|1);
        nxt.resize((m<<1)|1);
    }

    void addEdge(const int &u, const int &v) {
        nxt[++nE] = adj[u], go[adj[u] = nE] = v;
        nxt[++nE] = adj[v], go[adj[v] = nE] = u;
    }

    void augment(int u) {
        while (u) {
            int nu = mate[fa[u]];
            mate[mate[u] = fa[u]] = u;
            u = nu;
        }
    }

    int get_lca(int u, int v) {
        ++book_mark;
```

```
        while (true) {
            if (u) {
                if (book[u] == book_mark) return u;
                book[u] = book_mark;
                u = bel[fa[mate[u]]];
            }
            swap(u, v);
        }
    }

    void go_up(int u, int v, const int &mv) {
        while (bel[u] != mv) {
            fa[u] = v;
            v = mate[u];
            if (type[v] == 1) type[q[++q_n] = v] = 0;
            bel[u] = bel[v] = mv;
            u = fa[v];
        }
    }
    void after_go_up() {
        for (int u = 1; u <= n; ++u) bel[u] =
             bel[bel[u]];
    }

    bool match(const int &sv) {
        for (int u = 1; u <= n; ++u) bel[u] = u,
             type[u] = -1;
        type[q[q_n = 1] = sv] = 0;
        for (int i = 1; i <= q_n; ++i) {
            int u = q[i];
            for (int e = adj[u]; e; e = nxt[e]) {
                int v = go[e];
                if (!~type[v]) {
                    fa[v] = u, type[v] = 1;
                    int nu = mate[v];
                    if (!nu) {
                        augment(v);
                        return true;
                    }
                    type[q[++q_n] = nu] = 0;
                } else if (!type[v] && bel[u] != bel[v])
                     {
                    int lca = get_lca(u, v);
                    go_up(u, v, lca);
                    go_up(v, u, lca);
                    after_go_up();
                }
            }
        }
        return false;
    }

    void calc_max_match() {
        n_matches = 0;
        for (int u = 1; u <= n; ++u)
            if (!mate[u] && match(u)) ++n_matches;
    }
};

/*
int main() {
    int n,m;
    cin >> n >> m;
    edmond er(n, m);
    while(m--) {
        int x,y; cin >> x >> y;
        er.addEdge(x+1, y+1); // Input should be
             strictly 1-based indexed node.
    }
    er.calc_max_match();
    cout << er.n_matches << endl;
    for(int u = 1;u <= er.n; ++u)
        if(er.mate[u] > u) cout << er.mate[u]-1 << ' '
             << u-1 << '\n';
    return 0;
}
*/
```

5.3 tarjan_bridges_articulation_points

```
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> tin, low;
vector<pair<int,int>> bridge;
vector<int> cutpoint;
int timer;

void IS_BRIDGE(int v,int to) {
     bridge.push_back({v,to});
}

void dfs_bridge(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs_bridge(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v])
                IS_BRIDGE(v, to);
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs_bridge(i);
    }
}

void IS_CUTPOINT(int v) {
    cutpoint.push_back(v);
}

void dfs_points(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children=0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
```

```
        } else {
            dfs_points(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p!=-1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if(p == -1 && children > 1)
        IS_CUTPOINT(v);
}

void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs_points(i);
    }
}
```

5.4 min_cost_max_flow

```
typedef vector<int> vi;
bool ckmin(int &a, int b) {
  return b < a ? a = b, true : false;
}
bool ckmin(ll &a, ll b) {
  return b < a ? a = b, true : false;
}

struct MCMF {
  using F = int;
  using C = ll; // flow type, cost type
  struct Edge {
    int to;
    F flo, cap;
    C cost;
  };
  int N;
  vector<C> pi, dis;
  vi prev;
  vector<Edge> egde;
  vector<vi> g;
  void init(int _N) {
    N = _N;
    pi.resize(N), dis.resize(N), prev.resize(N),
      g.resize(N);
  }
  void addEdge(int u, int v, F cap, C cost) {
    assert(cap >= 0);
    g[u].pb(egde.size());
    egde.pb({v, 0, cap, cost});
    g[v].pb(egde.size());
    egde.pb({u, 0, 0, -cost});
  } // use asserts, don't try smth dumb
  bool path(int s, int t) { // find lowest cost path to
                            // send flow through
    const C inf = 1e18;
    for(int i = 0; i < N; i++)
      dis[i] = inf;
    using T = pair<C, int>;
    priority_queue<T, vector<T>, greater<T>> pq;
    pq.push({dis[s] = 0, s});
    while(pq.size()) { // Dijkstra
      T x = pq.top();
      pq.pop();
      if(x.first > dis[x.second]) continue;
      for(int e : g[x.second]) {
        const Edge &E = egde[e]; // all weights should
            be
                                 // non-negative
        if(E.flo < E.cap
           && ckmin(dis[E.to], x.first + E.cost
                              + pi[x.second]
                              - pi[E.to]))
          prev[E.to] = e, pq.push({dis[E.to], E.to});
      }
    } // if costs are doubles, add some EPS so you
    // don't traverse ~0-weight cycle repeatedly
    return dis[t] != inf; // return flow
  }
  void setpi() { // Call this function before calc if
                 // have -ve weights
    for(int i = 0; i < N; i++) {
      for(int e = 0; e < egde.size(); e++) {
        const Edge &E = egde[e]; // Bellman-Ford
        if(E.cap)
          ckmin(pi[E.to], pi[egde[e ^ 1].to] + E.cost);
      }
    }
  }
  pair<F, C> calc(int s, int t, int f) {
    assert(s != t);
    // setpi();
    F totFlow = 0;
    C totCost = 0;
    while(path(s, t)
          && f) { // p -> potentials for Dijkstra
      for(int i = 0; i < N; i++)
        pi[i] += dis[i]; // don't matter for
                         // unreachable nodes
      F df = f;
      for(int x = t; x != s; x = egde[prev[x] ^ 1].to) {
        const Edge &E = egde[prev[x]];
        ckmin(df, E.cap - E.flo);
      }
      f -= df;
      totFlow += df;
      totCost += (pi[t] - pi[s]) * df;
      for(int x = t; x != s; x = egde[prev[x] ^ 1].to)
        egde[prev[x]].flo += df, egde[prev[x] ^ 1].flo
            -= df;
    } // get max flow you can send along path
    return {totFlow, totCost};
  }
};
```

5.5 directed_eulerian_cycle

```
class DirectedEulerianCircuit{
    int n;
    vector<stack<int>> adj;
    int get(int u){
        return adj[u].top();
    }
public:
    DirectedEulerianCircuit(int _n):n(_n){
```

```
        adj.resize(n);
        for(int i=0;i<n;++i)
            adj[i].push(-1);
    }
    void addEdge(int u,int v){
        adj[u].push(v);
    }
    int findFirst(){
        int u = 0;
        for(int i=0;i<n;++i){
            if(adj[i].size()==1)continue;
            u = i;
            if(!(adj[i].size()&1))break;
        }
        return u;
    }
    vector<int> eulerCircuit(){
        stack<int> st;
        int u = 0;
        for(int i=0;i<n;++i){
            if(adj[i].size())u = i;
        }
        vector<int> circuit;
        st.push(u);
        while(!st.empty()){
            int x = get(st.top());
            if(x!=-1){
                adj[st.top()].pop();
                st.push(x);
            }
            else{
                circuit.push_back(st.top());
                st.pop();
            }
        }
        reverse(circuit.begin(),circuit.end());
        return circuit;
    }
};
```

5.6 hungarian

```
using lld = int; // value type
const lld inf = 1e18;
struct Hungarian {
  int n, m;
  vector<int> p;
  vector<lld> u, v, way;
  vector<vector<lld>> A;
  Hungarian(vector<vector<lld>> &arr) {
    n = arr.size() - 1;
    m = arr[0].size() - 1;
    A = arr;
    u.resize(n + 1);
    v.resize(m + 1);
    p.resize(m + 1);
    way.resize(m + 1);
  }
  void calc() {
    for(int i = 1; i <= n; ++i) {
      // cout << i << endl;
      p[0] = i;
      int j0 = 0;
      vector<lld> minv(m + 1, inf);
      vector<bool> used(m + 1, false);
      do {
        used[j0] = true;
        int i0 = p[j0], j1;
        lld delta = inf;
        for(int j = 1; j <= m; ++j) {
          if(!used[j]) {
            // cout << "T " << i0 << ' ' << j << endl;
            lld cur = A[i0][j] - u[i0] - v[j];
            if(cur < minv[j])
              minv[j] = cur, way[j] = j0;
            if(minv[j] < delta) delta = minv[j], j1 = j;
          }
        }
        for(int j = 0; j <= m; ++j)
          if(used[j]) u[p[j]] += delta, v[j] -= delta;
          else
            minv[j] -= delta;
        j0 = j1;
      } while(p[j0] != 0);
      do {
        int j1 = way[j0];
        p[j0] = p[j1];
        j0 = j1;
      } while(j0);
    }
  }

  vector<int> getAssignment() {
    vector<int> ans(n + 1);
    for(int j = 1; j <= m; ++j)
      ans[p[j]] = j;
    return ans;
  }

  lld mnCost() { return -v[0]; }
};
```

5.7 eulerian_cycle

```
class EulerianCircuit{
    int n,e;

    vector<vector<int>> adj;
    vector<bool> visit;
    vector<int> edges;
    int get(int u){
        while(visit[adj[u].back()])adj[u].pop_back();
        return adj[u].back();
    }
public:
    EulerianCircuit(int _n):n(_n),e(0){
        adj.resize(n);
        for(int i=0;i<n;++i)
            adj[i].push_back(0);
        edges.resize(2);
        visit.resize(2);
    }
    void addEdge(int u,int v){
        ++e;
        adj[u].push_back(e<<1);
        edges.push_back(v);
        adj[v].push_back((e<<1)|1);
        edges.push_back(u);
        visit.push_back(false);
        visit.push_back(false);
```

```
    }
    int findFirst(){
        int u = 0;
        for(int i=0;i<n;++i){
            if(adj[i].size()==1)continue;
            u = i;
            if(!(adj[i].size()&1))break;
        }
        return u;
    }
    vector<int> eulerCircuit(int u){
        if(!e)return vector<int>();
        stack<int> st;
        vector<int> circuit;
        st.push(u);
        while(!st.empty()){
            int x = get(st.top());
            if(x){
                visit[x] = visit[x^1] = true;
                st.push(edges[x]);
            }
            else{
                circuit.push_back(st.top());
                st.pop();
            }
        }
        return circuit;
    }
    vector<int> eulerCircuit(){
        if(!e)return vector<int>();
        return eulerCircuit(findFirst());
    }
};
```

5.8 edmond_blossom_weighted

```
using lld = int64_t;
#define all(v) v.begin(), v.end()
#define REP(i, l, r) for(int i = (l); i <= (r); ++i)
struct WeightGraph { // 1-based
  static const int inf = INT_MAX;
  struct edge {
    int u, v, w;
  };
  int n, nx;
  vector<int> lab;
  vector<vector<edge>> g;
  vector<int> slack, match, st, pa, S, vis;
  vector<vector<int>> flo, flo_from;
  queue<int> q;
  WeightGraph(int n_)
      : n(n_), nx(n * 2), lab(nx + 1),
       g(nx + 1, vector<edge>(nx + 1)), slack(nx + 1),
       flo(nx + 1),
       flo_from(nx + 1, vector(n + 1, 0)) {
    match = st = pa = S = vis = slack;
    REP(u, 1, n) REP(v, 1, n) g[u][v] = {u, v, 0};
  }
  int ED(edge e) {
    return lab[e.u] + lab[e.v] - g[e.u][e.v].w * 2;
  }
  void update_slack(int u, int x, int &s) {
    if(!s || ED(g[u][x]) < ED(g[s][x])) s = u;
  }
  void set_slack(int x) {
    slack[x] = 0;
    REP(u, 1, n)
    if(g[u][x].w > 0 && st[u] != x && S[st[u]] == 0)
      update_slack(u, x, slack[x]);
  }
  void q_push(int x) {
    if(x <= n) q.push(x);
    else
      for(int y : flo[x])
        q_push(y);
  }
  void set_st(int x, int b) {
    st[x] = b;
    if(x > n)
      for(int y : flo[x])
        set_st(y, b);
  }
  vector<int> split_flo(auto &f, int xr) {
    auto it = find(all(f), xr);
    if(auto pr = it - f.begin(); pr % 2 == 1)
      reverse(1 + all(f)), it = f.end() - pr;
    auto res = vector(f.begin(), it);
    return f.erase(f.begin(), it), res;
  }
  void set_match(int u, int v) {
    match[u] = g[u][v].v;
    if(u <= n) return;
    int xr = flo_from[u][g[u][v].u];
    auto &f = flo[u], z = split_flo(f, xr);
    REP(i, 0, int(z.size()) - 1)
    set_match(z[i], z[i ^ 1]);
    set_match(xr, v);
    f.insert(f.end(), all(z));
  }
  void augment(int u, int v) {
    for(;;) {
      int xnv = st[match[u]];
      set_match(u, v);
      if(!xnv) return;
      set_match(v = xnv, u = st[pa[xnv]]);
    }
  }
  /* SPLIT_HASH_HERE */
  int lca(int u, int v) {
    static int t = 0;
    ++t;
    for(++t; u || v; swap(u, v))
      if(u) {
        if(vis[u] == t) return u;
        vis[u] = t;
        u = st[match[u]];
        if(u) u = st[pa[u]];
      }
    return 0;
  }
  void add_blossom(int u, int o, int v) {
    int b = int(find(n + 1 + all(st), 0) - begin(st));
    lab[b] = 0, S[b] = 0;
    match[b] = match[o];
    vector<int> f = {o};
    for(int x : {u, v}) {
      for(int y; x != o; x = st[pa[y]])
        f.emplace_back(x),
          f.emplace_back(y = st[match[x]]), q_push(y);
      reverse(1 + all(f));
    }
    flo[b] = f;
```

```
  set_st(b, b);
  REP(x, 1, nx) g[b][x].w = g[x][b].w = 0;
  REP(x, 1, n) flo_from[b][x] = 0;
  for(int xs : flo[b]) {
    REP(x, 1, nx)
    if(g[b][x].w == 0 || ED(g[xs][x]) < ED(g[b][x]))
      g[b][x] = g[xs][x], g[x][b] = g[x][xs];
    REP(x, 1, n)
    if(flo_from[xs][x]) flo_from[b][x] = xs;
  }
  set_slack(b);
}
void expand_blossom(int b) {
  for(int x : flo[b])
    set_st(x, x);
  int xr = flo_from[b][g[b][pa[b]].u], xs = -1;
  for(int x : split_flo(flo[b], xr)) {
    if(xs == -1) {
      xs = x;
      continue;
    }
    pa[xs] = g[x][xs].u;
    S[xs] = 1, S[x] = 0;
    slack[xs] = 0;
    set_slack(x);
    q_push(x);
    xs = -1;
  }
  for(int x : flo[b])
    if(x == xr) S[x] = 1, pa[x] = pa[b];
    else
      S[x] = -1, set_slack(x);
  st[b] = 0;
}
bool on_found_edge(const edge &e) {
  if(int u = st[e.u], v = st[e.v]; S[v] == -1) {
    int nu = st[match[v]];
    pa[v] = e.u;
    S[v] = 1;
    slack[v] = slack[nu] = 0;
    S[nu] = 0;
    q_push(nu);
  } else if(S[v] == 0) {
    if(int o = lca(u, v)) add_blossom(u, o, v);
    else
      return augment(u, v), augment(v, u), true;
  }
  return false;
}
/* SPLIT_HASH_HERE */
bool matching() {
  for(auto &x : S)
    x = -1;
  for(auto &x : slack)
    x = 0;

  q = queue<int>();
  REP(x, 1, nx)
  if(st[x] == x && !match[x]) pa[x] = 0,
                              S[x] = 0, q_push(x);
  if(q.empty()) return false;
  for(;;) {
    while(q.size()) {
      int u = q.front();
      q.pop();
      if(S[st[u]] == 1) continue;
      REP(v, 1, n)
        if(g[u][v].w > 0 && st[u] != st[v]) {
          if(ED(g[u][v]) != 0)
            update_slack(u, st[v], slack[st[v]]);
          else if(on_found_edge(g[u][v]))
            return true;
        }
    }
    int d = inf;
    REP(b, n + 1, nx)
    if(st[b] == b && S[b] == 1) d
      = min(d, lab[b] / 2);
    REP(x, 1, nx)
    if(int s = slack[x]; st[x] == x && s && S[x] <= 0)
      d = min(d, ED(g[s][x]) / (S[x] + 2));
    REP(u, 1, n)
    if(S[st[u]] == 1) lab[u] += d;
    else if(S[st[u]] == 0) {
      if(lab[u] <= d) return false;
      lab[u] -= d;
    }
    REP(b, n + 1, nx)
    if(st[b] == b && S[b] >= 0)
      lab[b] += d * (2 - 4 * S[b]);
    REP(x, 1, nx)
    if(int s = slack[x]; st[x] == x && s && st[s] != x
                                  && ED(g[s][x]) == 0)
      if(on_found_edge(g[s][x])) return true;
    REP(b, n + 1, nx)
    if(st[b] == b && S[b] == 1 && lab[b] == 0)
      expand_blossom(b);
  }
  return false;
}
pair<lld, int> solve() {
  for(auto &x : match)
    x = 0;
  REP(u, 0, n) st[u] = u, flo[u].clear();
  int w_max = 0;
  REP(u, 1, n) REP(v, 1, n) {
    flo_from[u][v] = (u == v ? u : 0);
    w_max = max(w_max, g[u][v].w);
  }
  REP(u, 1, n) lab[u] = w_max;
  int n_matches = 0;
  lld tot_weight = 0;
  while(matching())
    ++n_matches;
  REP(u, 1, n)
  if(match[u] && match[u] < u) tot_weight
    += g[u][match[u]].w;
  return make_pair(tot_weight, n_matches);
}
void set_edge(int u, int v, int w) {
  g[u][v].w = g[v][u].w = w;
}
};
```

6 Geometry

6.1 points_inside_polygon

```
#include "../NumberTheory/inequality_solver.h"

struct pt{
    int x,y;
```

```cpp
    pt(int _x,int _y):x(_x),y(_y){}
    pt operator - (const pt &other) const{
        return pt(x-other.x,y-other.y);
    }
    pt operator - () const{
        return pt(-x,-y);
    }
};
ll cross(pt a,pt b){
    return a.x*1ll*b.y-a.y*1ll*b.x;
}
struct edge{
    pt s,e;
    bool isCounterClockWise(pt c,bool
         includeLinear=false){
        return cross(e-s,c-s)>=includeLinear;
    }
};

ll getPointsInsidePolygon(vector<edge> edges, bool
     includeOnLine = false){
    vector<inEq> inq;
    for(auto [s,e]:edges){
        pt t = e-s;
        inq.push_back(
            inEq(t.x,-t.y,cross(s,t)-1+includeOnLine)
        );
    }
    return integerInequalitySolver(inq);
}
```

6.2 polygon_line_cut

```cpp
template <class T>
bool isZero(T x) { return x == 0; }

const double EPS = 1e-9;
template <>
bool isZero(double x) { return fabs(x) < EPS; }

template <class T> struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x = 0, T y = 0) : x(x), y(y) {}

  bool operator<(P p) const { return tie(x, y) <
      tie(p.x, p.y); }
  bool operator==(P p) const { return isZero(x - p.x)
      && isZero(y - p.y); }

  P operator+(P p) const { return P(x + p.x, y + p.y); }
  P operator-(P p) const { return P(x - p.x, y - p.y); }
  P operator*(T d) const { return P(x * d, y * d); }
  P operator/(T d) const { return P(x / d, y / d); }

  T dot(P p) const { return x * p.x + y * p.y; }
  T cross(P p) const { return x * p.y - y * p.x; }
  T cross(P a, P b) const { return (a - *this).cross(b
      - *this); }
};

// Line Intersection
template <class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (isZero(d)) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}

// Polygon Cut
typedef Point<double> P;
vector<P> polygonCut(const vector<P> &poly, P s, P e) {
  if (poly.size() <= 2) return {};
  vector<P> res;
  for (size_t i = 0; i < poly.size(); i++) {
    P cur = poly[i], prev = i ? poly[i - 1] :
         poly.back();
    if (isZero(s.cross(e, cur))) {
      res.push_back(cur);
      continue;
    }
    bool side = s.cross(e, cur) < 0;
    if (side != (s.cross(e, prev) < 0))
      res.push_back(lineInter(s, e, cur, prev).second);
    if (side)
      res.push_back(cur);
  }
  return res;
}

// Polygon Area, returns twice the actual area, signed.
template <class T>
T polygonArea2(const vector<Point<T>> &v) {
  T a = v.back().cross(v[0]);
  for (size_t i = 0; i + 1 < v.size(); i++)
    a += v[i].cross(v[i + 1]);
  return a;
}
```

6.3 lichao

```cpp
// Important : Here range l,r denotes [l,r). With this
     a bug is avoided which is if we take [l,r] then
     when both l and r are negative the update [l,mid]
     can result in infinite
//          recursion. E.g: l = -7, r = -6. mid =
     (l+r)/2 = -6 so [l,mid] = [-7,-6], resulting in
     infinite recursion.

typedef long long ll;
struct line {
    ll m, c;
    ll operator()(ll x) { return m * x + c; }
};


class Lichao {
    vector<line> lc_tree;
public:
    Lichao(int N) {
        lc_tree = vector<line>(N << 1, { 0, -(ll)(1e18)
             });
    }

    void insert(int v, int l, int r, line cur) {
        if (l + 1 == r) {
            if (lc_tree[v](l) < cur(l)) lc_tree[v] = cur;
            return;
```

```
        }
        int mid = (l + r) >> 1;
        int z = (mid - l) << 1;
        if (lc_tree[v].m > cur.m) swap(lc_tree[v], cur);
        if (cur(mid) > lc_tree[v](mid)) {
            swap(lc_tree[v], cur);
            insert(v + 1, l, mid, cur);
        }
        else
            insert(v + z, mid, r, cur);
    }


    ll query(int v, int l, int r, int pt) {   // Finding
         maximum value of function at x = pt
        if (l + 1 == r) return lc_tree[v](pt);
        int mid = (l + r) >> 1;
        int z = (mid - l) << 1;
        if (pt <= mid) return max(lc_tree[v](pt),
             query(v + 1, l, mid, pt));
        else return max(lc_tree[v](pt), query(v + z,
             mid, r, pt));
    }
};
```

6.4 convex_hull

```
#include<bits/stdc++.h>
using namespace std;

struct pt {
    double x, y;
    pt(double x=0, double y=0): x(x),y(y) {}
    bool operator == (const pt &rhs) const {
        return (x == rhs.x && y == rhs.y);
    }
    bool operator < (const pt &rhs) const {
        return y < rhs.y || (y==rhs.y && x < rhs.x);
    }
    bool operator > (const pt &rhs) const {
        return y > rhs.y || (y==rhs.y && x > rhs.x);
    }
};

double area(const vector<pt> &poly) {
    int n = static_cast<int>(poly.size());
    double area = 0;
    for(int i=0;i<n;++i) {
        pt p = i ? poly[i-1] :poly.back();
        pt q = poly[i];

        area += p.x * q.y - p.y * q.x;
    }
    area = fabs(area)/2;
    return area ;
}


int orientation(pt a, pt b, pt c) {
    double v = a.x*(b.y-c.y)+
              b.x*(c.y-a.y)+c.x*(a.y-b.y);
    if (v < 0) return -1; // clockwise
    if (v > 0) return +1; // counter-clockwise
    return 0;
}

bool cw(
    pt a, pt b, pt c, bool include_collinear=false) {
    int o = orientation(a, b, c);
    return o < 0 || (include_collinear && o == 0);
}
bool collinear(pt a, pt b, pt c) {
    return orientation(a, b, c) == 0;
}

// Returns square of distance between point a and b
long double square_dist(pt a, pt b) {
    return (a.x-b.x)*1ll*(a.x-b.x)
        + (a.y-b.y)*1ll*(a.y-b.y);
}

// Returns points in clokwise order with a[0] as
// left lowermost point(Minimum point)
void convex_hull(
    vector<pt>& a, bool include_collinear = false) {
    pt p0 = *min_element(a.begin(), a.end(),
        [](pt a,pt b) {
        return make_pair(a.y, a.x)
               < make_pair(b.y, b.x);
    });
    auto square_dist_from_p0 = [&p0](pt& a){
        return square_dist(p0,a);
    };
    sort(a.begin(), a.end(), [&p0](
        const pt& a, const pt& b) {
        int o = orientation(p0, a, b);
        if (o == 0)
            return square_dist_from_p0(a)
               < square_dist_from_p0(b);
        return o < 0;
    });
    if (include_collinear) {
        int i = (int)a.size()-1;
        while (i >= 0 &&
            collinear(p0, a[i], a.back())) i--;
        reverse(a.begin()+i+1, a.end());
    }

    vector<pt> st;
    for (int i = 0; i < (int)a.size(); i++) {
        while (st.size() > 1 && !cw(st[st.size()-2],
             st.back(), a[i], include_collinear))
            st.pop_back();
        st.push_back(a[i]);
    }


    a = st;
}

// poly: Contains points of polygon in counter
// clockwise order, with poly[0] as lower
// leftmost point(minimum point) and top is the index
// of top rightmost point(maximum point).
// Min/Max are defined by comparator operator
// defined for points.
// It returns 1: Point outside, 0: Point in boundary,
// -1: Point inside.
int pointVsConvexPolygon(
    pt &point, vector<pt> &poly, int top) {
    if(point < poly[0] || point > poly[top]) return 1;
    int o = orientation(point, poly[top], poly[0]);
    if(o == 0) {
```

```
        if(point == poly[0] || point == poly[top])
            return 0;
        return (top == 1 || top+1==poly.size())
            ? 0 : -1;
    } else if(o < 0) {
        auto itLeft = upper_bound(
            poly.rbegin(), poly.rend() - top-1, point);
        return orientation((itLeft == poly.rbegin() ?
            poly[0]:itLeft[-1]), itLeft[0], point);
    } else {
        auto itRight = lower_bound(poly.begin()+1,
                        poly.begin()+top,point);
        return orientation(point,
            itRight[0], itRight[-1]);
    }
}



// a and b represent direction:
// Returns 1 if direction is ccw, 0 is coolinear
// and -1 is direction is cw (clockwise).
int ccw(const pt &a, const pt&b) {
    long double ans = (long double)a.x*b.y
                    - (long double)a.y*b.x;
    if(ans < 0) return -1;
    return (ans > 0);
}

// Maximum distance (squared) between
// given sets of points
// in O(N) or O(N*log(N))
// (first make them a convex polygon)
long double maxSquareDist(vector<pt> &poly) {
    int n = static_cast<int>(poly.size());
    long double res = 0;
    for(int i = 0, j = n < 2 ? 0 : 1; i < j; ++i)
        for(;; j = (j+1)%n) {
            res = max(res, square_dist(
                poly[i], poly[j]));
            pt dir1 = pt(poly[i+1].x-poly[i].x,
                poly[i+1].y-poly[i].y);
            pt dir2 = pt(poly[(j+1)%n].x-poly[j].x,
                poly[(j+1)%n].y-poly[j].y);
            if(ccw(dir1, dir2) <= 0) break;
        }
    return res;
}
```

6.5 3d

```
using ll = long long;
using ld = long double;
using uint = unsigned int;
template<typename T>
using pair2 = pair<T, T>;
using pii = pair<int, int>;
using pli = pair<ll, int>;
using pll = pair<ll, ll>;

#define pb push_back
#define mp make_pair
#define all(x) (x).begin(),(x).end()
#define fi first
#define se second

const ld eps = 1e-8;
bool eq(ld x, ld y) {
        return fabsl(x - y) < eps;
}
bool ls(ld x, ld y) {
        return x < y && !eq(x, y);
}
bool lseq(ld x, ld y) {
        return x < y || eq(x, y);
}

ld readLD() {
        int x;
        scanf("%d", &x);
        return x;
}

struct Point {
        ld x, y, z;

        Point() : x(), y(), z() {}
        Point(ld _x, ld _y, ld _z) : x(_x), y(_y),
            z(_z) {}

        void scan() {
                x = readLD();
                y = readLD();
                z = readLD();
        }

        Point operator + (const Point &a) const {
                return Point(x + a.x, y + a.y, z + a.z);
        }
        Point operator - (const Point &a) const {
                return Point(x - a.x, y - a.y, z - a.z);
        }
        Point operator * (const ld &k) const {
                return Point(x * k, y * k, z * k);
        }
        Point operator / (const ld &k) const {
                return Point(x / k, y / k, z / k);
        }
        ld operator % (const Point &a) const {
                return x * a.x + y * a.y + z * a.z;
        }
        Point operator * (const Point &a) const {
                return Point(
                        y * a.z - z * a.y,
                        z * a.x - x * a.z,
                        x * a.y - y * a.x
                        );
        }
        ld sqrLen() const {
                return *this % *this;
        }
        ld len() const {
                return sqrtl(sqrLen());
        }
        Point norm() const {
                return *this / len();
        }
};

ld ANS;
//triangle contains 4 points, 4th point is a copy of 1st
Point A[4], B[4];
```

```
// P lies on plane, n is perpendicular, return A' such
// that AA' is parallel plane,
// PA' is perpendicular
Point getHToPlane(Point P, Point A, Point n) {
        n = n.norm();
        return P + n * ((A - P) % n);
}

// Checks if point P is in traingle t
bool inTriang(Point P, Point* t) {
        ld S = 0;
        S += ((t[1] - t[0]) * (t[2] - t[0])).len();
        for (int i = 0; i < 3; i++)
                S -= ((t[i] - P) * (t[i + 1] - P)).len();
        return eq(S, 0);
}
// Assuming A,B,C are on same line,
// it finds if C is between B
bool onSegm(Point A, Point B, Point C) {
        return lseq((A - B) % (C - B), 0);
}
//A is a point on line, a is direction of line, B is
// point on plane,
//b is normal to plane, l is used to store result in
// case intersection exists
bool intersectLinePlane(Point A, Point a, Point B,
     Point b, Point &I) {
        if (eq(a % b, 0)) return false;
        ld t = ((B - A) % b) / (a % b);
        I = A + a * t;
        return true;
}
//A is point on line, a is direction of line
//returns projection of P on line
Point getHToLine(Point P, Point A, Point a) {
        a = a.norm();
        return A + a * ((P - A) % a);
}
//get minimum distance of A from PQ
ld getPointSegmDist(Point A, Point P, Point Q) {
        Point H = getHToLine(A, P, Q - P);
        if (onSegm(P, H, Q)) return (A - H).len();
        return min((A - P).len(), (A - Q).len());
}
//
ld getSegmDist(Point A, Point B, Point C, Point D) {
        ld res = min(
                min(getPointSegmDist(A, C, D),
                        getPointSegmDist(B, C, D)),
                min(getPointSegmDist(C, A, B),
                        getPointSegmDist(D, A, B)));
        Point n = (B - A) * (D - C);
        if (eq(n.len(), 0)) return res;
        n = n.norm();
        Point I1, I2;
        if (!intersectLinePlane(A, B - A, C,
                (D - C) * n, I1)) throw;
        if (!intersectLinePlane(C, D - C, A,
                (B - A) * n, I2)) throw;
        if (!onSegm(A, I1, B)) return res;
        if (!onSegm(C, I2, D)) return res;
        return min(res, (I1 - I2).len());
}
```

7 Polynomials

7.1 fft

```
#define vi vector<int>
using i64 = int64_t;
typedef complex<double> cd;
constexpr int MAXN = 2e5 + 10;
int rev[MAXN * 3];
const cd I(0, 1);
const double PI = acos(-1);
cd F[MAXN * 3];

void fft(int N, int sgn = 1) {
    int bit = __lg(N);
    for (int i = 0; i < N; ++i) {
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (bit
            - 1));
        if (i < rev[i])
            swap(F[i], F[rev[i]]);
    }
    for (int l = 1; l < N; l <<= 1) {
        cd step = exp(sgn * PI / l * I);
        for (int i = 0; i < N; i += l * 2) {
            cd cur(1, 0);
            for (int k = i; k < i + l; ++k) {
                cd g = F[k], h = F[k + l] * cur;
                F[k] = g + h, F[k + l] = g - h;
                cur *= step;
            }
        }
    }
}

vi convolution(vi A, vi B) {
    int n = A.size() - 1, m = B.size() - 1;
    int N = 1 << __lg(n + m + 1) + 1;
    for (int i = 0; i <= N; ++i)
        F[i] = cd(i <= n ? A[i] : 0, i <= m ? B[i] : 0);
    fft(N);
    for (int i = 0; i <= N; ++i)
        F[i] = F[i] * F[i];
    fft(N, -1);
    vi C(n + m + 1);

    for (int i = 0; i <= n + m; ++i) {
        C[i] = int64_t(round(F[i].imag() / (N * 2))) %
            1009;
    }
    return C;
}
```

7.2 multipoint

```
void trim(vi &a) {
  while(a.size() && !a.back())
    a.pop_back();
}
int eval(vi &p, int pt) {
  int ans = 0;
  for(int i = 0, b = 1; i < p.size();
      i++, b = (b * 1ll * pt) % M)
    ans = (ans + (b * 1ll * p[i]) % M) % M;
  return ans;
}
```

```
vi eval(vi &p, vi &pts) {
  cap = false;
  int n = pts.size();
  vi tree[4 * n + 1];
  function<void(int, int, int)> build =
    [&](int v, int l, int r) {
      if(l == r) {
        tree[v] = {(M - pts[l]) % M, 1};
      } else {
        int m = (l + r) / 2;
        build(2 * v, l, m), build(2 * v + 1, m + 1, r);
        tree[v] = tree[2 * v] * tree[2 * v + 1];
      }
    };
  build(1, 0, n - 1);
  vi ans(n, 0);
  function<void(int, int, int, vi)> evaluate
    = [&](int v, int l, int r, vi curr) {
        trim(curr);
        if(r - l <= 32) {
          for(int i = l; i <= r; i++) {
            ans[i] = eval(curr, pts[i]);
          }
        } else {
          int m = (l + r) / 2;
          evaluate(2 * v, l, m, curr % tree[2 * v]);
          evaluate(2 * v + 1, m + 1, r,
                   curr % tree[2 * v + 1]);
        }
      };
  evaluate(1, 0, n - 1, p % tree[1]);
  cap = true;
  return ans;
}
```

7.3 poly_mono

```
typedef long long ll;
const int P1 = 880803841, G1 = 26;//(105*2^23)+1
const int P2 = 897581057, G2 = 3;//(107*2^23)+1
const int P3 = 998244353, G3 = 3;//(119*2^23)+1

typedef vector<int> vi;
#define pb push_back
#define f(n) for(int i=0;i<(n);i++)

const int M = 998244353; // Also works on - 880803841
    and 897581057
int expo(int a,int b,int mod=M){int ans=1;
    while(b){if(b&1) ans=(ans*1ll*a)%mod;
    a=(a*1ll*a)%mod; b>>=1;} return ans;}
int mod_div(int a,int b,int mod=M){return
    (a*1ll*expo(b,mod-2))%mod;}

const int root=62; // For 998244353. NOT SURE->
    Otherwise while(expo(root,M/2)==1) root++;
void ntt(vi &a){
        int n=a.size(),L=31-__builtin_clz(n);
        static vi rt(2,1);

        for(static int k=2,s=2;k<n;k*=2,s++) {
                rt.resize(n);
                int z[]={1,expo(root,M>>s)};
                for(int i=k;i<2*k;i++)
                    rt[i]=(ll)rt[i/2]*z[i&1]%M;
        }
        vi rev(n);
        f(n){
                rev[i]=(rev[i/2]|(i&1)<<L)/2;
                if(i<rev[i]) swap(a[i],a[rev[i]]);
        }
        for(int k=1;k<n;k*=2)
                for(int i=0;i<n;i+=2*k)
                        for(int j=0;j<k;j++){
                                int
                                    z=(ll)rt[j+k]*a[i+j+k]%M,&ai=
                                a[i+j+k]=ai-z+(z>ai?M:0);
                                ai+=(ai+z>=M?z-M:z);
                        }
}
bool cap=true; // Set to FALSE to get n+m-1 size
    product. Set to TRUE for Exp() and below Operations
#define ao(a) f(a.size()) cout<<((2*a[i]<M)?
    a[i]:a[i]-M)<<" ";
vi brutemul(const vi &a,const vi &b){
        int n=a.size(),m=b.size(),s=n+m-(cap?
            min(n,m):1);
        vi out(s,0);
        f(n) for(int j=0;j<min(m,s-i);j++)
            out[i+j]=(out[i+j]+(a[i]*1ll*b[j]%M))%M;
        return out;
}
vi mul(const vi &a,const vi &b){
        if(a.empty() || b.empty()) return {};
        int p=a.size(),q=b.size();
        if(min(p,q)<32) return brutemul(a,b);
        int s=p+q-(cap?
            min(p,q):1),n=1<<(32-__builtin_clz(s));
        int inv=mod_div(1,n);
    vi L(a),R(b),out(n);
        L.resize(n); R.resize(n);
        ntt(L); ntt(R);
        f(n) out[-i&(n-1)]=(ll)L[i]*R[i]%M*inv%M;
        ntt(out);
        return {out.begin(),out.begin()+s};
}

#define opr operator
vi& opr*=(vi& a,const vi &b){return a=mul(a,b);}
vi opr*(const vi& a,const vi &b){return mul(a,b);}
vi opr*(const int k,const vi &P){vi Q(P); for(int &a:Q)
    a=(a*1ll*k)%M; return Q;}
vi& opr*=(vi& P,const int k){for(int &a:P)
    a=(a*1ll*k)%M; return P;}
vi& opr+=(vi& a,const vi &b){
    if(a.size()<b.size()) a.resize(b.size());
    for(int i=0;i<b.size();++i){
        a[i]+=b[i];
        if(a[i]>=M) a[i]-=M;
    }
    return a;
}
vi opr+(const vi& a,const vi &b){vi c(a); return c+=b;}
vi& opr-=(vi& a,const vi &b){
    if(a.size()<b.size()) a.resize(b.size());
    for(int i=0;i<b.size();++i){
        a[i]-=b[i];
        if(a[i]<0) a[i]+=M;
    return a;
```

```
}
vi opr-(const vi& a,const vi &b){vi c(a); return c-=b;}
vi opr/(const int k,const vi &P){ // P[0] should be
    non-zero
        vi Q{mod_div(1,P[0])};
        int n=P.size();
        for(int k=2;Q.size()<n;k*=2){
                Q*=(vi{2}-Q*vi(P.begin(),P.begin()+k));
                Q.resize(k);
        }
        Q*=k;
        return Q;
}
vi& opr/=(vi& a,const vi &b){return a*=1/b;}
vi opr/(const vi& a,const vi &b){vi c(a); return c/=b;}
int degree(const vi &a) {
  int n = a.size();
  while(n && !a[n-1]) n--;
  return n;
}
vi& opr%=(vi &a,const vi &b){
        if(a.empty()) return a;
        int deg=degree(a)-degree(b)+1;
        if(deg<=0) return a;
        vi X(b.rbegin(),b.rend()); X.resize(deg);
        vi Y(a.rbegin(),a.rend()); Y.resize(deg);
        if(deg&(deg-1)) X.resize(1<<(__lg(deg)+1));
        vi Q=Y/X; Q.resize(deg);
        reverse(Q.begin(),Q.end());
        a-=Q*b; a.pop_back();
        while(a.size() && !a.back()) a.pop_back();
        return a;
}
vi opr%(const vi &a,const vi &b){vi v=a; return v%=b;}

// All P[i] should be in range [0,M)
// P.size() should be a power of 2.
void deriv(vi &P){
    f(P.size()-1) P[i]=(P[i+1]*1ll*(i+1))%M;
    P.pop_back();
}

const int N=1e5+1,N1=1<<(__lg(N)+1);
int inv_mod[N1+1]; // Only required for below Poly
    functions
void IM(){f(N1) inv_mod[i+1]=(mod_div(1,i+1));}

void Integrate(vi &P,int C){ // C is integration
    constant =: P[0]
        P.pb(0);
    for(int i=P.size()-1;i>=0;i--)
         P[i+1]=(P[i]*1ll*inv_mod[i+1])%M;
    P[0]=C%M;
}
void Log(vi &P){ // P[0] should be 1
        vi invP=1/P;
    deriv(P);
    P*=invP;
    Integrate(P,0);
    P.resize(invP.size());
}
// NOTE:- Set cap=true, for functions below or use
    Q.resize(k) at the end of loop of Exp().
void Exp(vi &P){ // P[0] should be 0
        int n=P.size();
        vi Q={1};
        for(int k=2;Q.size()<n;k*=2){
                vi lnQ(Q);
                lnQ.resize(k);
                Log(lnQ);
                Q*=(vi(P.begin(),P.begin()+k)+vi{1}-lnQ);
        }
        swap(P,Q);
}
void Power(vi &P,int k){
    int n=P.size(),C=P[0],shift=0;
    if(!C){
        while(shift<n && !P[shift]) shift++;
        if(shift*k>=n){
                memset(&P[0],0,n*sizeof(P[0]));
                        return;
                }
                P=vi(P.begin()+shift,P.end()-shift*(k-1));
        C=P[0];
    }
    if(C!=1){
        P*=mod_div(1,C);
        C=expo(C,k);
    }
    Log(P);
        P*=k;
    Exp(P);
    if(C!=1) for(int &a:P) a=(a*1ll*C)%M;
    if(shift){
        vi Q(n,0);
        f(P.size()) Q[i+shift*k]=P[i];
        swap(P,Q);
    }
}
void Root(vi &P,int k){ // P[0] should be 1
        Log(P);
        P*=mod_div(1,k);
        Exp(P);
}
```

8 Theory

8.1 Taylor function approximation

$$y_{n+1} = y_n + [A(x) - g(y)/g'(y_n)]$$

8.2 Pentagonal Theorem

$$\prod_{n=1}^{\infty}(1-x^n) = \sum_{k=-\infty}^{\infty} (-1)^k \, x^{k(3k-1)/2}$$

$$= 1 + \sum_{k=1}^{\infty}(-1)^k \left(x^{k(3k+1)/2} + x^{k(3k-1)/2}\right)$$

8.3 AP Polynomial Evaluation

$$let\ degree(p(x)) = n;$$

$$\exists\ g(x)\ with\ d(g(x)) = n+1$$

$$such\ that\ e^x g(x) = \sum_{0}^{\infty}(p(i)x^i/i!)$$

8.4 Lagrange Inversion Theorem

$$let f(g(x)) = x, f(0) = g(0) = 0,$$
$$f'(0) = g'(0) = 1;$$

$$[x^n]H(f(x)) = [x^{n-1}]\frac{1}{n}\frac{H'(x)}{((\frac{g(x)}{x})^n)}$$
$$[x^n]H(f(x)) = [x^{n-2}]\frac{H(x)}{((\frac{g(x)}{x})^{n-1})}(\frac{g'(x)}{g(x)^2})$$

8.5 Chirp Z Transform

$$ij = {}^{i+j}C_2 - {}^{i}C_2 - {}^{j}C_2$$

8.6 Mobius Transform

$$\mu(p^k) = [k == 0] - [k == 1]$$

$$\sum_{d|n} \mu(d) = [n == 1]$$

8.7 Convolution

Xor Convolution

$$b_j = \sum a_i(-1)^{bitcount(i\&j)}$$
$$inverse\ is\ the\ same$$

Or Convolution

$$b_j = \sum a_i[i\&j == i]$$

And Convolution

$$b_j = \sum a_i[i|j == i]$$

GCD Convolution

$$b_j = \sum_{j|i} a_i;\ inverse:\ reverse\ of\ code$$

LCM Convolution

$$b_j = \sum_{i|j} a_j;\ inverse:\ reverse\ of\ code$$

8.8 Picks Theorem

$$A = i + (b/2) - 1$$

8.9 Euler's Formula

$$F + V = E + 2$$

8.10 DP Optimisations

Knuth Dp Optimisation

$$dp(i,j) = \min_{i \le k \le j}[dp(i,k) + dp(k+1,j) + C(i,j)]$$

$$use\ for\ finding\ opt(i,j)$$
$$opt(i,j-1) \le opt(i,j) \le opt(i+1,j);$$

$$conditions:$$
$$a \le b \le c \le d$$
$$\implies C(b,c) \le C(a,d)$$
$$\implies C(a,c) + C(b,d) \le C(a,d) + C(b,c)$$

```
for l in [0..n] do
    for i in [0..n] do
        L ← opt(i, i + l − 1)
        R ← opt(i + 1, i + l − 2)
        for k in [L..R] do
            dp(i, i + l) ← ...
        end for
    end for
end for
```

Convex Hull Trick

$$dp_i = \min_{j \le i}[dp_j + b_j \cdot a_i]$$

$$conditions:$$
$$b[j] \ge b[j+1]$$
$$a[i] \le a[i+1]$$

add (b_j , dp_j) = (m,c) as $y = mx + c$, in Convex Hull

Divide and Conquer

$$dp(i,j) = \min_{0 \le k \le j} dp(i-1,k-1) + C(k,j)$$

conditions :
$j < 0 \implies dp(i,j) = 0$
$opt(i,j) \le opt(i,j+1)$
special case : (for which above holds)
$\implies C(a,c) + C(b,d) \le C(a,d) + C(b,c)$

```
Function compute(l, r, optl, optr):
if l > r then
    return
end if
mid ← ⌊(l+r)/2⌋
best ← (∞, −1)
for k ← optl to min(mid, optr) do
    cost ← (k > 0?dp_before[k − 1] : 0) + C(k, mid)
    best ← min(best, (cost, k))
```

end for
$dp\_cur[mid] \leftarrow best.first$
$opt \leftarrow best.second$
`compute`$(l, mid-1, optl, opt)$
`compute`$(mid+1, r, opt, optr)$

Function `solve`():
`dp_before` $\leftarrow \{0\}_{i=0}^{n-1}$
`dp_cur` $\leftarrow \{0\}_{i=0}^{n-1}$
for $i \leftarrow 0$ to $n-1$ **do**
    `dp_before`$[i] \leftarrow C(0, i)$
end for
for $i \leftarrow 1$ to $m-1$ **do**
    `compute`$(0, n-1, 0, n-1)$
    `dp_before` $\leftarrow$ `dp_cur`
end for
return `dp_before`$[n-1]$

Aliens Trick

Ensure convexity/concavity in answer array

$$\forall i \in \{1, 2, \ldots, n\}, \quad \text{ans}[i] - \text{ans}[i-1] \leq \text{ans}[i+1] - \text{ans}[i]$$
$$\forall i \in \{1, 2, \ldots, n\}, \quad \text{ans}[i] - \text{ans}[i-1] \geq \text{ans}[i+1] - \text{ans}[i]$$